Assignment 6 - Advanced Programming 2026

Anton Bang Lejbølle & Jonas Dennis Hansen

October 2026

1 Introduction

Briefly mention general concerns, any ambiguities in the problem text and how you resolved them, and your own estimation of the quality of your solution.

For this assignment, we generally found the problem text quite clear, the only small ambiguity we found was if the name of the stopped workers should be saved, and thereby if new workers are allowed to have the same name as a stopped worker. We chose to resolve it by not saving the name of stopped workers, such that they are completely removed, and new workers are therefore allowed to have the same name as a stopped worker.

We are confident in the quality of our solution, and during the implementation we have generally followed the hints from the problem text.

One small detail of our timeout implementation, is that we did not extend WorkerMsg, as we just reused the WMsgWorkerJobCancel message from our job cancellation implementation, in order to tell the worker to cancel the job.

One improvement we could have made, was to be able to better handle if a worker thread crashes. This could have been handled by making the workers continuously send a ping to the SPC, and for the SPC to remove/restart a worker if it does not receive a ping from a Worker for a prolonged time.

In general our solution is functional, and we have made test cases that cover the five tasks given in the problem text: Adding Workers, Job Cancellation, Timeouts, Exceptions and Removing Workers. Our tests cover duplicate worker names, jobs queued before a worker is added, multiple jobs, job status and waiting, and if workers can be reused after cancellation, timeouts or crashes. We also test stopping workers multiple times and reusing worker names after removal.

2 Answers to questions

1. What happens when a job is enqueued in an SPC instance with no workers, and a worker is added later? Which of your tests demonstrate this?

If a job is enqueued in an SPC instance with no workers, then the job will be pending. When a worker is added later, then this worker will pick up the pending job.

The test that demonstrates this is called "job queued before worker is added".

2. Did you decide to implement timeouts centrally in SPC, or decentrally in the worker threads? What are the pros and cons of your approach? Is there any observable effect?

We decided to implement timeouts centrally in the SPC. One pro is that even if the worker thread crashes then the SPC can still handle timeouts. Another pro is that implementing it centrally only requires one thread. So by handling it centrally we ensure that as long as the SPC thread is running then it will correctly handle timeouts for all jobs.

A con is that the SPC only checks for timeouts every time checkTimeouts is called, and since a MsgTick is only sent once every second, the SPC might only check for timeouts once every second. So some jobs might run for up to a second more than specified in jobMaxSeconds.

3. Which of your tests verify that if a worker executes a job that is cancelled, times out, or crashes, then that worker can still be used to execute further jobs?

The test called "worker reusable after cancellation" verify that if a worker executes a job that is cancelled then that worker can still be used to execute further jobs
The test called "worker reusable after timeout" verify that if a worker executes a job that is timed out then that worker can still be used to execute further jobs
The test called "worker reusable after crash" verify that if a worker executes a job that crashes then that worker can still be used to execute further jobs

4. If a worker thread were somehow to crash (either due to a bug in the worker thread logic, or because a scoundrel killThreads it), how would that impact the functionality of the rest of SPC?

If a worker thread crashes, the rest of SPC and the other workers will continue running because the central SPC thread and all the worker threads run independently.

Our implementation does not detect or remove a worker that crashes unexpectedly. The worker will therefore remain registered as idle or busy while it can no longer handle messages. If the SPC assigns a job to this worker the job will never finish and clients waiting for it will have to wait until the timeout is detected. Therefore a worker crash does not stop the entire system, but it can leave jobs unfinished and reduce the available worker capacity, as one worker will never finish any jobs assigned to it.

5. Suppose a client calls jobCancel (which is asynchronous) and then immediately jobStatus. Can we expect this always produces DoneCancelled, or is there a risk of race condition?

When a client calls jobCancel a MsgJobCancel is sent to the SPC. As soon as this message is handled then the jobid is put into spcJobsDone with the reason DoneCancelled. If a client then immediatly calls jobStatus, this will only produce something other than DoneCancelled, if the SPC handles another message that puts the jobid into spcJobsDone with another reason before handling the MsgJobCancel.

Assuming that the SPC has not received a message that sets the job to be done before receiving the MsgJobCancel, then it will always produce DoneCancelled.